Assignment 1: Basic

第一次实践作业：BPE 与 Transformer

参考 Stanford CS336 Spring 2026 <https://cs336.stanford.edu/> 与 Assignment 1 <https://github.com/stanford-cs336/assignment1-basics/tree/main> （官方仓库的PDF文件可以完整查看具体作业详情）

发布时间	9.21
测试时间（分配资源）	10.19
作业类型	个人作业
open_web_text	作为训练与评估数据集，获取途径： wget https://huggingface.co/datasets/stanford-cs336/owt-sample/resolve/main/owt_train.txt.gz <https://huggingface.co/datasets/stanford-cs336/owt-sample/resolve/main/owt_train.txt.gz> gunzip owt_train.txt.gz wget https://huggingface.co/datasets/stanford-cs336/owt-sample/resolve/main/owt_valid.txt.gz <https://huggingface.co/datasets/stanford-cs336/owt-sample/resolve/main/owt_valid.txt.gz> gunzip owt_valid.txt.gz
BPE 正式实验	使用 `owt_train.txt` 训练，词表上限 32,000（包含特殊 token）；特殊 token 通过 `special_tokens` 列表自行配置； 在 `owt_valid.txt` 上评估，展示编码后的前 200 个 token 和完整验证耗时
Transformer 正式实验	使用 `owt_train.txt` 训练； 在 `owt_valid.txt` 上计算评分 PPL； ！！任务二必须使用本人在任务一实现并训练的 BPE 分词器、词表、merges 和特殊 token 配置
TinyStories	可用于调试，不作为正式评分数据，不与 OWT 结果混排

训练预算	每人拥有单张 A100/A800 80GB 的 24 小时使用额度。**BPE 不另设训练时长限制；6 小时预算仅适用于 Transformer**：从程序启动开始，到训练、保存最后模型和最终验证完成，累计不超过 6 小时；续训累计计时，不重新计算预算
提交格式	W&B 实验链接，每次实验必须在 W&B 中打印完整源码，并保存源码快照和评分结果，方便助教抽查源码正确性
BPE 评分（10分）	结果正确性 5 分、完整验证效率 5 分
Transformer 评分（10分）	按 PPL 评估模型效果，共 10 分；采用预算结束前**最后**保存的模型

作业详情：

一、环境与数据

数据与评分

使用上表指定的 OWT 训练集和验证集，公共数据只读。W&B 只记录评分和源码审查需要的内容：源码、BPE 分词结果及完整验证耗时、Transformer 最终验证 PPL 和低频训练 PPL 曲线。

开发机环境配置

使用 uv 管理环境（暂时不用 miniconda）。登录开发机，将项目复制到个人目录；公共原始数据只读，小样本存入自己的工作目录：

```
cp -r /data/nlp_course/assignment1-basics ~/assignment1-basics
cd ~/assignment1-basics
# 尚未安装 uv 时执行
curl -LsSf https://astral.sh/uv/install.sh | sh
# 若提示 uv: command not found
export PATH="$HOME/.local/bin:$PATH"
```

使用 Python 3.13，以避免自动安装的 Python 版本与 PyTorch 不兼容：

```
uv python install 3.13
uv python pin 3.13
uv venv .venv
uv sync
uv run python -V
```

依赖下载困难时，可在 `pyproject.toml` 的 `[tool.uv]` 下使用镜像配置：

```
[tool.uv]
index-url = "https://pypi.tuna.tsinghua.edu.cn/simple"
extra-index-url = ["https://mirrors.aliyun.com/pypi/simple/"]
```

W&B版本锁定：

```
# pyproject.toml：合并到已有 [project]，不要重复声明该表。
[project]
dependencies = [
    # 保留原有全部依赖，在现有数组内新增下面这一项。
    "wandb==0.30.0",  # 记录器已做过离线验证的版本。
]

```

```
# 在自己的项目目录中，完成上面的 TOML 修改后执行
uv lock
uv sync --locked
uv run python -c "import wandb; print(wandb.__version__)"

# 后续复现实验时，使用自己已生成的 uv.lock
uv sync --locked
```

自行实现的范围

- 允许 Python 标准库、`regex`、NumPy、PyTorch 基础张量运算和自动微分，以及 W&B、pytest。
- 允许 `nn.Parameter`、`nn.Module/ModuleList/Sequential` 等容器、`torch.optim.Optimizer` 基类、`torch.nn.init.trunc_normal_` 和模型保存加载。
- 不得用现成 Tokenizer、`nn.Linear/Embedding/Transformer`、现成注意力、归一化、激活、softmax、交叉熵、AdamW、调度器或梯度裁剪替代手写组件。`torch.nn.functional` 的现成实现同样不得使用。
- 不使用他人完整实现或 AI 生成核心作业代码；可查概念和基础 API。测试依赖的参考包只能用于测试。

建议保留 `bpe.py`、`train_bpe.py`、`transformer_main.py`、`transformer_module.py`，也可自行拆分。所有自编源码放在项目目录内；不能放到 `outputs`、数据目录或虚拟环境等被记录器排除的位置。官方测试通过 `tests/adapters.py` 对接，不修改测试预期。

W&B：一次配置，两项任务共用

在项目根目录执行，每次新开终端重新设置环境变量：

```
uv run wandb login
export WANDB_ENTITY="你的实际entity"
export WANDB_PROJECT="lmfs-assignment1"
export WANDB_MODE="online"
export PYTHONIOENCODING="utf-8"
export STUDENT_ID="你的学号"
```

将下面完整代码保存为项目根目录的 `wandb_record.py`，两项任务直接导入。

所有本人实现的源码须放在项目目录内；不要放在 `outputs`、数据目录或虚拟环境等排除目录。源文件须为 UTF-8。运行中不要修改算法代码。

```python
from contextlib import contextmanager
from pathlib import Path
import hashlib
import os
import uuid
import wandb


@contextmanager
def experiment(task, config, root="."):
    root = Path(root).resolve()
    skipped = {".git", ".venv", "venv", "outputs", "wandb", "data",
               "datasets", "checkpoints", "__pycache__", ".pytest_cache"}
    suffixes = {".py", ".sh", ".ipynb", ".cu", ".c", ".cpp", ".h", ".rs"}
    sources = []
    for directory, dirs, names in os.walk(root):
        dirs[:] = sorted(d for d in dirs if d not in skipped)
        for name in sorted(names):
            path = Path(directory) / name
            rel = path.relative_to(root)
            if (path.suffix in suffixes
                or name in {"pyproject.toml", "uv.lock", ".python-version"}
                or (rel.parts[0] == "configs" and path.suffix in {".json", ".
                sources.append(path)
    with wandb.init(
        entity=os.environ["WANDB_ENTITY"],
        project=os.environ.get("WANDB_PROJECT", "lmfs-assignment1"),
        id=uuid.uuid4().hex[:12], resume="never",
        name=f"{os.environ['STUDENT_ID']}-{task}",
        group=os.environ["STUDENT_ID"], job_type=task, config=config,
        settings=wandb.Settings(console="wrap"),
    ) as run:
        out = root / "outputs" / run.id
        out.mkdir(parents=True, exist_ok=True)
        artifact = wandb.Artifact(f"source-{run.id}", type="source-code")
        dump, hashes = [], {}
        for path in sources:
            rel = path.relative_to(root).as_posix()
            content = path.read_bytes()
            digest = hashlib.sha256(content).hexdigest()
            hashes[path] = digest
            frozen = out / "source" / rel
            frozen.parent.mkdir(parents=True, exist_ok=True)
            frozen.write_bytes(content)
            artifact.add_file(str(frozen), name=rel)
            lines = "\n".join(f"{i:05d} | {line}" for i, line in
                              enumerate(content.decode("utf-8-sig").splitline
            block = f"===== SOURCE {rel} SHA256={digest} =====\n{lines}\n"
            print(block, flush=True)
            dump.append(block)
```

5481字

```
        source_dump = out / "source_dump.txt"
        source_dump.write_text("\n".join(dump), encoding="utf-8")
        artifact.add_file(str(source_dump), name="source_dump.txt")
        run.log_artifact(artifact)
        run.save(str(source_dump), base_path=str(out), policy="now")
        try:
            yield run
        finally:
            changed = [str(path.relative_to(root)) for path, digest in hashes
                       if not path.is_file()
                       or hashlib.sha256(path.read_bytes()).hexdigest() != di
            run.summary["source_changed"] = changed
            if changed:
                print("WARNING: 运行中源码发生变化: ", changed, flush=True)
```

下面两项任务分别提供运行与记录方法。W&B 的 `run` 是上传对象，任务涉及的训练、编码或验证仍需自行实现。

二、任务一：字节级 BPE（10 分）

实现要求

1. 初始词表包含全部 256 种字节，文本采用 UTF-8；实现 GPT-2 风格预分词，保留空格、换行和标点。
2. 统计相邻符号对频次，反复合并最高频对；同频时选择 bytes 元组字典序最大的对。保存合并顺序，不能跨预分词或特殊 token 边界合并。
3. `vocab_size` 为最终词表上限，包含基础字节、特殊 token 和合并项；编码返回整数 ID，并按训练的合并顺序应用规则。
4. 特殊 token 映射为单独 ID；解码先拼接 bytes，再以 UTF-8 解码（非法序列用 `errors="replace"`）。正常输入必须精确还原，不能忽略空白差异。
5. 支持保存加载和惰性的 `encode_iterable()`；流式处理须正确处理边界，不能任意截断后独立编码。

预分词使用以下正则，可调用 `regex.finditer`：

```
PAT = r"""'(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|
```

特殊 token 配置

接口与 CS336 一致，接收学生指定的 `special_tokens: list[str]`，不要将实现写死为只识别某一个标记。下方配置中的 `["<|endoftext|>"]` 是示例，请替换为本人实际使用的完整清单，并由 W&B 的 config 记录。

- 每个特殊 token 都须加入词表并映射为单独 ID，即使它未出现在训练文本中；这些条目计入 32,000 的词表上限。
- 训练时先隔离特殊 token，不将其纳入普通符号对统计，也不能跨越其边界合并；编码时将其整体识别为一个 token，解码时恢复原始标记。
- `<|endoftext|>` 是官方实验使用的文档分隔标记。本课程允许自行配置特殊 token 清单；如增加 BOS、EOS、PAD 等标记，应说明用途。仅将标记加入清单不会自动向文本插入它。
- 保存和加载须保留完整清单及对应 ID；任务二沿用任务一的配置。

上面的 `PAT` 是普通文本的预分词正则，**不是特殊 token 清单**。若使用正则匹配特殊 token，须转义其中的正则字符；存在包含关系的标记应优先匹配较长者。

参考 CS336 官方讲义第 2.4–2.6 节 <https://github.com/stanford-cs336/assignment1-basics/blob/main/cs336_assignment1_basics.pdf>：接口支持传入特殊 token 列表，官方 OWT 实验指定包含 `<|endoftext|>`；本课程在此基础上允许学生自行配置。

接口格式

```python
from collections.abc import Iterable, Iterator

class BPE:
    def __init__(self, vocab: dict[int, bytes] | None = None,
                 merges: list[tuple[bytes, bytes]] | None = None,
                 special_tokens: list[str] | None = None):
        raise NotImplementedError

    def train(self, input_path: str, vocab_size: int,
              special_tokens: list[str]) -> None:
        # 训练后设置 self.vocab 和 self.merges，并重建编码查找表。
        raise NotImplementedError

    def encode(self, text: str) -> list[int]:
        raise NotImplementedError

    def encode_iterable(self, iterable: Iterable[str]) -> Iterator[int]:
        raise NotImplementedError

    def decode(self, ids: list[int]) -> str:
        raise NotImplementedError

    def save(self, path_prefix: str) -> None:
        # 词表、merges 分开保存习惯，并另存特殊 token 等配置。
        raise NotImplementedError

    def load(self, path_prefix: str) -> None:
        raise NotImplementedError

    @classmethod
    def from_files(cls, vocab_filepath: str, merges_filepath: str,
                   special_tokens: list[str] | None = None):
        raise NotImplementedError
```

官方 `run_train_bpe(input_path, vocab_size, special_tokens, kwargs)` 适配函数应调用你的训练方法并返回 `(vocab, merges)`；`get_tokenizer(vocab, merges, special_tokens)` 返回具备上述编码解码能力的实例。

保存时 bytes 不能直接交给 JSON：可使用 hex/base64 并明确还原规则，或保留可信的 pickle；不要用 `str(bytes)` 充当可逆序列化。恢复后 vocab、merges、特殊 token 和编码行为必须一致。

评分与记录要求

评分项	分值	W&B 必须展示
正确性	5	验证文本编码后的前 200 个 token、全文还原结果，以及本人源码

完整验证效率	5	完整 `owt_valid.txt` 的完整验证耗时（全文 encode、全文 decode、原文比较），按耗时排序分档给分

效率计时包含完整验证文本的 `encode` 、完整结果的 `decode` 以及与原文比较的总耗时。读取文件、加载分词器在计时前完成；前 200 个 token 详情、表格生成及上传在计时后完成。**不以 BPE 训练耗时、前 200 个 token 的耗时或整个 W&B run 时长评分。** 每次正式计时使用刚加载的分词器，不提前编码验证集建立缓存；同一数据、相同 CPU/进程资源条件下比较，错误结果不按速度优势计分。

Unicode、连续空白、特殊 token 和保存加载等可自行测试，不要求逐条上传；无需记录每一次合并、内存、吞吐或其他中间数据。

W&B 配置代码：保存为 `train_bpe.py`

下面是完整入口。学生实现 `bpe.py` 中的 `BPE` 类。 `save()` 和 `load()` 使用同一个路径前缀，须包含恢复特殊 token 行为所需的信息。

```python
import json
import time
from pathlib import Path
import wandb
from wandb_record import experiment


def main():
    config = {
        "train_file": "/data/nlp_course/1-basic-data/owt_train.txt",
        "valid_file": "/data/nlp_course/1-basic-data/owt_valid.txt",
        "vocab_size": 32000,
        "special_tokens": ["<|endoftext|>"],  # 示例：替换为本人实际清单
    }
    with experiment("bpe", config) as run:
        from bpe import BPE
        bpe = BPE()
        bpe.train(config["train_file"], config["vocab_size"], config["special
        prefix = str(Path("outputs") / run.id / "tokenizer")
        bpe.save(prefix)
        # 重新加载，避免使用训练或先前评估产生的编码缓存。
        bpe = BPE()
        bpe.load(prefix)
        with open(config["valid_file"], encoding="utf-8", newline="") as f:
            text = f.read()
        started = time.perf_counter()
        ids = bpe.encode(text)
        decoded_text = bpe.decode(ids)
        passed = decoded_text == text
        valid_seconds = time.perf_counter() - started
        # 以下均为展示/审查操作，不计入验证耗时。
        raw = text.encode("utf-8")
        first_ids = ids[:200]
        rows, offset = [], 0
        for i, token_id in enumerate(first_ids):
            piece = bpe.vocab[token_id]
            if not isinstance(piece, bytes):
                raise TypeError("vocab 的值必须是 bytes")
            end = offset + len(piece)
            try:
                piece.decode("utf-8", errors="strict")
                standalone_valid = True
            except UnicodeDecodeError:
                standalone_valid = False
            rows.append([i, int(token_id), piece.hex(), repr(piece),
                         repr(bpe.decode([token_id])), standalone_valid,
                         offset, end, piece == raw[offset:end]])
            offset = end
        columns = ["position", "token_id", "token_hex", "token_bytes",
                   "decoded_token", "standalone_utf8_valid",
```

```
                    "byte_start", "byte_end", "matches_input_bytes"]
        print("前 200 个 encode ID: ", first_ids, flush=True)
        print("前 200 个 token 整体 decode: ", repr(bpe.decode(first_ids)), fl
        print("逐 token 详情字段: ", columns, flush=True)
        for row in rows:
            print(row, flush=True)
        run.log({"bpe/first_200_tokens": wandb.Table(columns=columns, data=ro
        result = {"bpe/valid_seconds": valid_seconds,
                  "bpe/valid_token_count": len(ids), "bpe/roundtrip_passed":
                  "bpe/vocab_size": len(bpe.vocab)}
        run.summary.update(result)
        print(json.dumps(result, ensure_ascii=False), flush=True)
        print("任务二加载本人 BPE 的路径前缀: ", prefix, flush=True)
        if len(ids) < 200 or not passed:
            raise ValueError("评估文本不足 200 个 token 或还原失败，请检查输入与


if __name__ == "__main__":
    main()
```

运行命令：

```
uv run python train_bpe.py
```

在 W&B 的 Table 查看前 200 个 token 的 ID、原始字节、逐项解码结果、字节位置（左闭右开）及原文匹配情况；Logs 同时展示前 200 个 ID 和这些 token 的整体解码结果。Summary 中 `bpe/valid_seconds` 为完整验证耗时，`bpe/roundtrip_passed` 表示全文解码是否与原文完全一致。

单个 token 可能只是汉字或 emoji 的部分 UTF-8 字节，单独解码出现 `�` 不一定是错误；应先拼接 bytes 再解码，不能拼接逐 token 解码的字符串。完整还原文本也不代表合并规则正确，助教仍结合源码审查。

三、任务二：Transformer（10 分）

实现要求

自行实现以下组件，具体公式和初始化要求参考 CS336 讲义：

- Linear、Embedding、RMSNorm、SiLU/SwiGLU。
- RoPE（作用于 Q/K）、稳定 softmax、因果多头自注意力。
- Pre-norm Transformer block、残差连接、完整语言模型及词表输出。
- 稳定交叉熵、AdamW、warmup + cosine 学习率、全局梯度裁剪。
- 数据采样、训练/验证循环、模型和优化器保存恢复。

允许用 `loss.backward()` 自动计算梯度，但前向计算和参数更新规则须自己实现。输出 logits 形状为 `(batch_size, sequence_length, vocab_size)`。

使用任务一的 BPE

任务二必须使用本人任务一的 BPE、词表、merges 和特殊 token 配置，编码也调用本人实现。训练和验证使用同一套分词器，开始训练后不要更换词表。词表 ID 从 0 连续编号，模型词表维度使用实际 `len(bpe.vocab)` 。

将任务一终端打印的路径前缀用于 `BPE.load()` 。不调用现成 Tokenizer 或加载预训练模型。模型从随机权重开始；保留原始文本。若按自己的设计额外插入 BOS/EOS 等标记，须使用任务一已配置的特殊 token，并说明插入位置、规则及这些位置是否参与验证损失计算；不得在任务二临时更换词表或特殊 token ID。

评分与记录要求

- 正式评分：6 小时预算内最后保存模型在指定 OWT 验证集上的 PPL，共 10 分；记录最终 step，以及从程序启动到训练、保存和最终验证完成的实际墙钟耗时，用于核对预算。
- 训练曲线：每 100 次参数更新记录一次训练 PPL，最后一个不足 100 步的区间也可记录一次。不记录每个 batch 的全文输出或参数，不上传中间 checkpoint。
- 源码：启动时自动打印并保存，包括 BPE、模型、优化器及验证代码。

PPL 为 `exp(总目标 token NLL / 有效目标 token 数)` 。验证使用真实文本，关闭梯度并切换 eval 模式；不能用生成文本概率代替，也不能平均各 batch 的 PPL。全班使用同一验证集，分词正确性需结合任务一核查。训练 PPL 用于看过程，评分使用最终验证 PPL。

模型和分词产物自行保存在本地，暂不要求在 W&B 上传模型、额外生成样例、资源曲线或测试明细。保留本地文件供复查及续训。

6 小时计时

从 `transformer_main.py` 开始执行就计时。导入模块、连接 W&B、打印源码、加载/编码数据、初始化模型、训练、保存最后模型和最终验证均计入；最后的汇总上传及进程退出不计入。每次运行的起止 UTC 时间和耗时均记录到 W&B。

文首规定为 **不超过 6 小时（21,600 秒）** 。W&B 同时记录距 6 小时的差值：负数表示提前结束，正数表示超时。计时记录本身不会自动停止训练，学生须在循环中检查剩余时间，并为保存及最终验证预留时间。

将下面三行放在 `transformer_main.py` **最顶部、导入 PyTorch 等模块之前**；文件若有 `from __future__` ，将它放在这三行之前：

```
import time
PROGRAM_STARTED_UNIX = time.time()
PROGRAM_STARTED_CLOCK = time.perf_counter()
```

续训时，将下文 `previous_elapsed_seconds` 填为此前所有关联 run 的实际累计耗时，并填写 `resume_from_run`，不得清零。首次从零训练保持 0 和 null（Python 中为 `None`）。

W&B 接入

1. 在 `transformer_main.py` 定义自己的训练函数。 它负责加载本人 BPE、编码数据、训练、保存最后模型及最终验证，保持以下接口：

```
def train_model(config, run):
    # 自行实现；run 仅用于下面第 2 步的日志调用。
    raise NotImplementedError
```

2. 在自己的训练循环中加入以下计数和记录。 `loss_sum` 和 `num_tokens` 替换成当前微批次中实际的 NLL 总和与有效目标 token 数；数值在梯度累积缩放之前计算。`global_step` 为已经完成的 optimizer 更新次数。损失与 token 数在损失所在设备上累积，仅每 100 次参数更新记录时取回 CPU，避免每个微批次为日志调用 `.item()`。

```
# 训练循环开始前初始化
interval_nll = None
interval_tokens = None

# 每个微批次计算损失后累计；detach() 避免保留计算图。
# 独立的累计张量不会修改原始 loss_sum，也不参与反向传播。
if interval_nll is None:
    interval_nll = torch.zeros((), device=loss_sum.device, dtype=torch.float6
    interval_tokens = torch.zeros((), device=loss_sum.device, dtype=torch.int
interval_nll.add_(loss_sum.detach().to(dtype=torch.float64))
interval_tokens.add_(torch.as_tensor(
    num_tokens, device=loss_sum.device, dtype=torch.int64).detach())

# 每完成一次 optimizer 更新后检查，不放在每个微批次更新位置。
if global_step % 100 == 0:
    # 仅在记录时取回累计值；num_tokens 可为整数或设备上的标量张量。
    logged_nll = interval_nll.item()
    logged_tokens = interval_tokens.item()
    if logged_tokens > 0:
        run.log({"train/ppl": math.exp(logged_nll / logged_tokens),
                 "train/step": int(global_step),
                 "time/elapsed_seconds": config["previous_elapsed_seconds"]
                     + time.perf_counter() - PROGRAM_STARTED_CLOCK})
    interval_nll.zero_()
    interval_tokens.zero_()
```

如果自己的损失函数返回均值 `loss`，则记录用的 NLL 总和为 `loss.detach() * num_tokens`，不要在每个微批次调用 `.item()`。文件顶部导入 `torch` 和 `math`。日志中的训练 PPL 仍是最近一个记录区间的结果；如需记录最后不足 100 步的区间，使用相同的取值、记录及清零逻辑。

3. 在 `train_model()` 最后返回最终验证结果；在文件底部添加启动代码。

保存最后模型后，对该模型完成验证；使用 GPU 时，在 `train_model()` 返回前调用 `torch.cuda.synchronize()`，确保 GPU 工作完成后再结束计时。`val_nll_sum` 为整个验证过程的目标 NLL 总和，`val_token_count` 为对应 token 数。下列右侧变量替换为本人代码中的同义变量；返回字段名保持不变。

```
# 放在 train_model() 末尾
return {
    "val_nll_sum": float(val_nll_sum),
    "val_token_count": int(val_token_count),
    "step": int(global_step),
}
```

下面入口按程序起点自动计时，并在正常返回或 Python 异常时记录。最终 PPL 仍由下面入口计算和上传。

```python
# 放在 transformer_main.py 文件底部；train_model 必须已在上方定义。
import json
import math
from datetime import datetime, timezone
from wandb_record import experiment

if __name__ == "__main__":
    config = {
        "train_file": "/data/nlp_course/1-basic-data/owt_train.txt",
        "valid_file": "/data/nlp_course/1-basic-data/owt_valid.txt",
        "bpe_prefix": "outputs/替换为任务一run_id/tokenizer",
        "max_train_seconds": 21600,
        "previous_elapsed_seconds": 0.0,
        "resume_from_run": None,
        # 将本人实际使用的模型、优化器等训练参数也填在这里。
    }
    with experiment("transformer", config) as run:
        run.define_metric("train/step")
        run.define_metric("train/ppl", step_metric="train/step")
        def utc(timestamp):
            return datetime.fromtimestamp(timestamp, timezone.utc).isoformat(

        previous = float(config["previous_elapsed_seconds"])
        if not math.isfinite(previous) or not 0 <= previous < 21600:
            raise ValueError("此前累计耗时无效或已用完预算")
        run.summary["time/program_started_utc"] = utc(PROGRAM_STARTED_UNIX)
        run.summary["time/previous_elapsed_seconds"] = previous
        # 训练循环可调用：remaining = config["deadline_clock"] - time.perf_cour
        config["deadline_clock"] = PROGRAM_STARTED_CLOCK + 21600 - previous
        try:
            result = train_model(config, run)
        finally:
            # 正常返回或 Python 异常时都记录；强制 kill/断电无法保证执行。
            ended_unix = time.time()
            current_seconds = time.perf_counter() - PROGRAM_STARTED_CLOCK
            total_seconds = previous + current_seconds
            timing = {
                "time/program_started_utc": utc(PROGRAM_STARTED_UNIX),
                "time/experiment_ended_utc": utc(ended_unix),
                "time/current_run_seconds": current_seconds,
                "time/total_seconds": total_seconds,
                "time/difference_from_6h_seconds": total_seconds - 21600,
                "time/within_6h": total_seconds <= 21600,
            }
            run.summary.update(timing)
            print(json.dumps(timing, ensure_ascii=False), flush=True)
        if result["val_token_count"] <= 0:
            raise ValueError("验证 token 数必须大于零")
        val_loss = result["val_nll_sum"] / result["val_token_count"]
        if not math.isfinite(val_loss) or val_loss < 0:
```

```
            raise ValueError("验证平均 NLL 非有限值或为负，请检查计算")
        try:
            final_ppl = math.exp(val_loss)
        except OverflowError as error:
            raise ValueError("验证 PPL 溢出，请检查模型数值") from error
        if not math.isfinite(final_ppl):
            raise ValueError("PPL 非有限值，请检查模型数值")
        final = {"val/ppl": final_ppl,
                 "val/token_count": result["val_token_count"],
                 "train/final_step": result["step"],
                 "train/total_seconds": total_seconds}
        run.summary.update(final)
        print(json.dumps(final, ensure_ascii=False), flush=True)
        if not 0 <= total_seconds <= 21600:
            raise ValueError("累计训练耗时超过预算或无效，请核查")
```

运行命令：

```
uv run python transformer_main.py
```

在 W&B Charts 查看 `train/ppl`，在 Summary 查看最终 `val/ppl`、`time/total_seconds`、`time/difference_from_6h_seconds` 和 `time/within_6h`。异常强杀时可能只有最近一次低频记录的 `time/elapsed_seconds`，不能把它当作完整结束时间。

不要在 `train_model()` 内再次调用 `wandb.init()`，一次正式实验只对应一个 run。

四、提交

每人提交一份md文档，写明：姓名、学号及 BPE、Transformer 两个正式 W&B run 链接；续训时附关联 run。

确认助教能查看完整源码、BPE 前 200 token 详情与完整验证耗时、Transformer 最终验证 PPL 和训练曲线。

若网络中断，可设 `WANDB_MODE=offline`，恢复网络后执行 `uv run wandb sync /实际离线run目录`；确认远端内容可访问再提交。W&B 日志用于审阅，不代替助教对源码和结果真实性的核查。